

Python for **Absolute Beginners**

From zero programming experience to writing and shipping your own Python programs.

13

Sections

Setup to classes, in order

295+

Concepts

Each one is a demo you can run

60+

Practice Problems

You write the code

60+

Workshop Problems

A bigger task per step

optional, for more practice

2

Real Projects

Built with you, line by line

No experience needed

We install Python together, then start at **print()**. Nothing is assumed.

Every solution included

Try it yourself first, then compare with the **full solution file**.

Every line on screen

You watch each line being **typed and run**. Nothing is pasted in already finished.

Your Python Journey

32h 25m
OF VIDEO

13 sections, 295+ concepts and 2 real projects. Every concept taught on screen, line by line.

Part 1 Foundations



Part 2 Leveling Up



Python Keywords: 30 of 35 by the End

Python has only 35 keywords, and you will have written 30 of them yourself by the end of this course. The five in grey (assert, del, yield, async, await) and the four soft keywords are for later courses.

Values

TrueFalseNone

Logic & Membership

andornotinis

Conditionals

ifelifelse

Loops

forwhilebreakcontinue

Functions

defreturnlambda

Classes

class

Error Handling

tryexceptfinallyraiseargassert

Modules & Imports

importfromas

Scope

globalnonlocal

Context Managers

with

Cleanup & Flow

passdel

Generators & Async

yieldasyncawait

Soft Keywords

matchcase_underscoretype

30 / 35

KEYWORDS COVERED

keyword

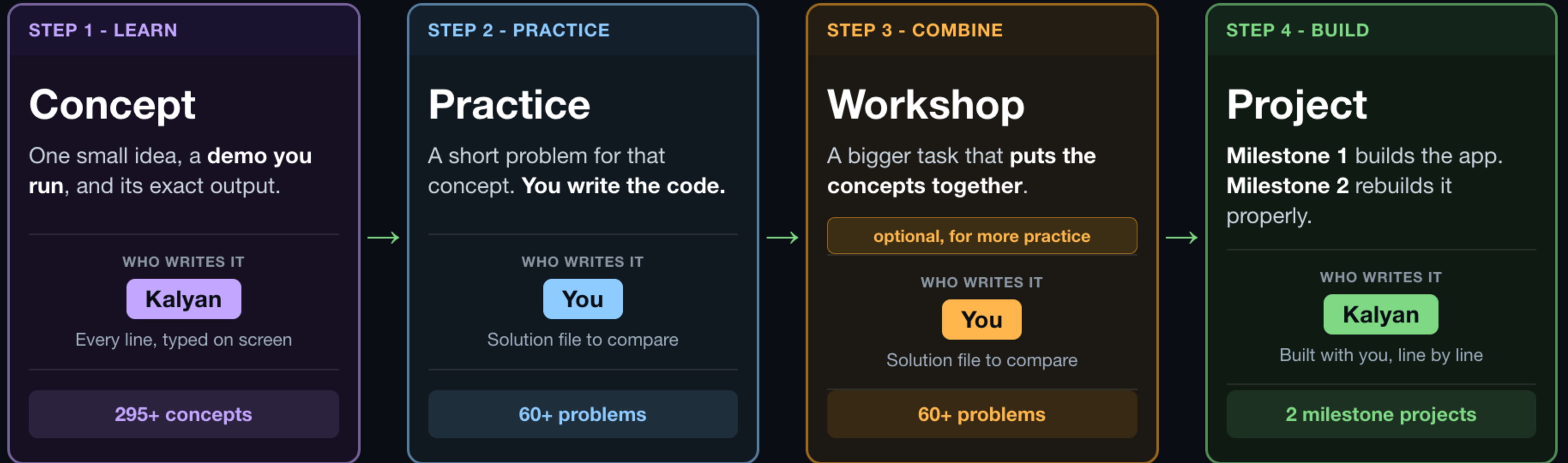
You write this in the course

keyword

Not covered here

How You **Learn** Here

The same rhythm in every section, and it ends in a **real project**, not an exercise.



TRY FIRST, THEN CHECK Every practice and workshop problem ships with a **full solution file**. Attempt it yourself, then compare.

Running the **Concept Demos**

Every concept is a few lines you can run. **Two ways to get them onto your machine.**

Way 1 From the course website

- 1 Open the section page and find the concept
- 2 **Copy** the code block, it is one click
- 3 Paste it into that section's `sNN_lecture.py`
- 4 **Run it** and compare with the output on the page

WEBSITE BOOKMARK IT

<https://python-for-beginners.stacksimplify.com>

Way 2 From the GitHub repo

- 1 Clone `python-for-absolute-beginners` once
- 2 Open the section's `python-learn/` folder
- 3 Every step is a file: `s05_step01_lists.py`
- 4 **Run it as it is**, nothing to copy

REPO BOOKMARK IT

<https://github.com/stacksimplify/python-for-absolute-beginners>

WHICH ONE

Typing it yourself sticks better, so **Way 1 while you learn**. Keep the repo open as the finished reference, and for the days you just want to run something fast.

Practice and **Workshop** Problems

Same two routes as the demos, but here the point is that **you write the code first**.

Way 1 On the course website

Question The task and its **expected output**

Hint Stuck? Open **Need a hint?** for a nudge, not the answer

Solution Open **Show the solution** only after you have tried

Everything is on one page, so nothing to download to get started.

WEBSITE **BOOKMARK IT**

<https://python-for-beginners.stacksimplify.com>

Way 2 In the GitHub repo

Question file `python-practice/s05_practice01_lists.py`

You write The file holds **the task as comments**, and empty space below

Solution file `solutions/s05_practice01_sol_lists.py`

Workshops work the same way, under `python-workshop/`.

REPO **BOOKMARK IT**

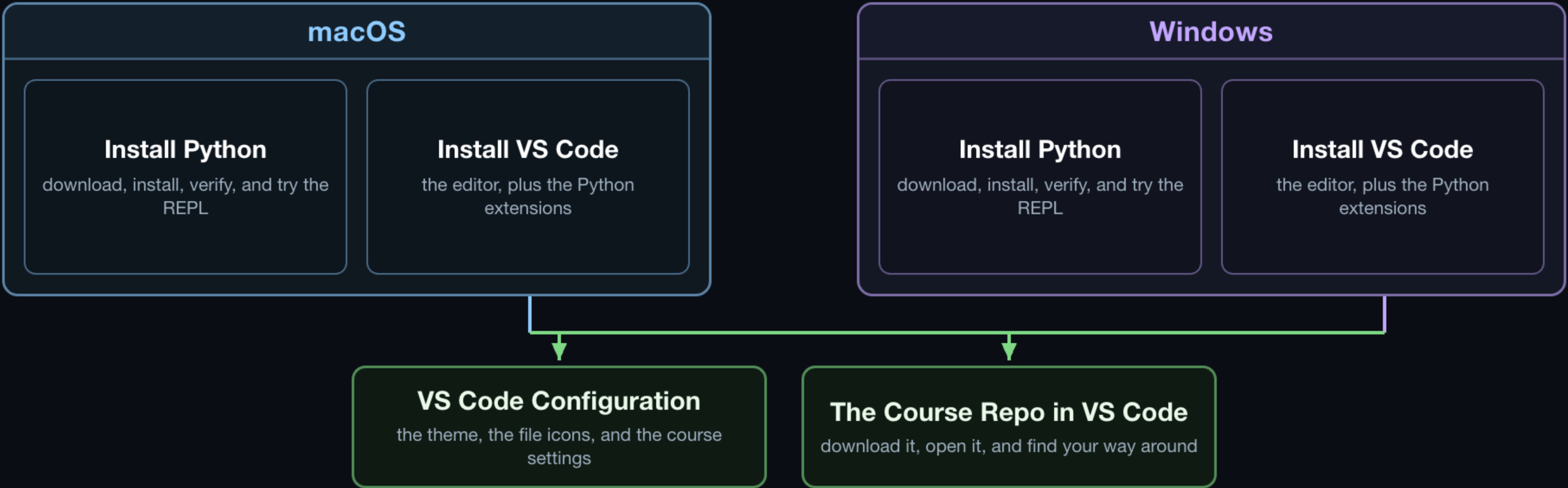
<https://github.com/stacksimplify/python-for-absolute-beginners>

THE ONE RULE

The solution is always one click or one folder away, which makes it easy to peek. **Write your own version first, even a wrong one.** That attempt is what makes the solution stick.

Section 01: Python Environment Setup

Pick your operating system, install two things, then everyone meets again for the same two steps.



WATCH THE VIDEOS FOR YOUR OWN OS

On **macOS**, follow the macOS install videos. On **Windows**, follow the Windows ones. From **VS Code Configuration** onward, every video is the same for everyone.

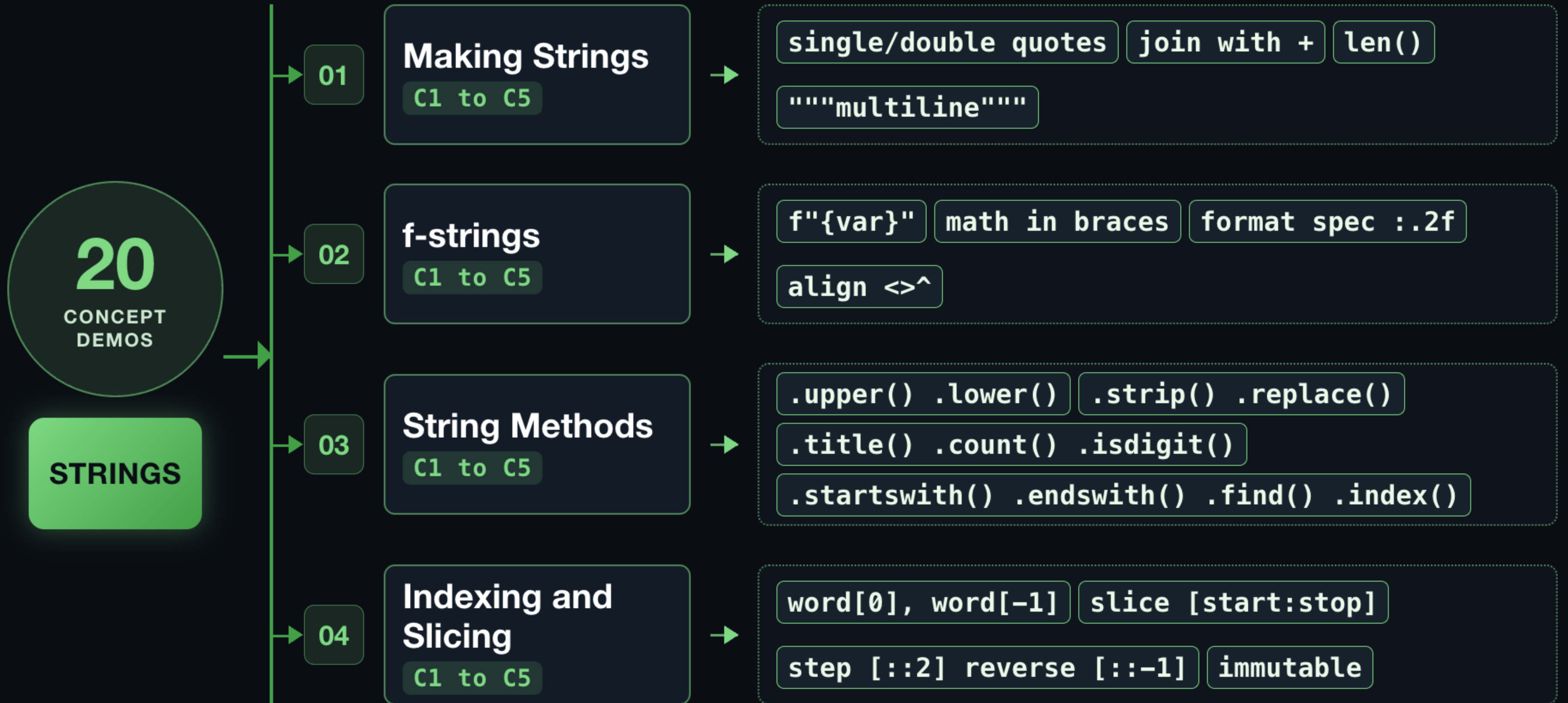
Section 02: Variables and Data Types Concept Map

The full path through this section - we open here before each concept, then dive in.



Section 03: Strings and f-strings **Concept Map**

The full path through this section - we open here before each concept, then dive in.



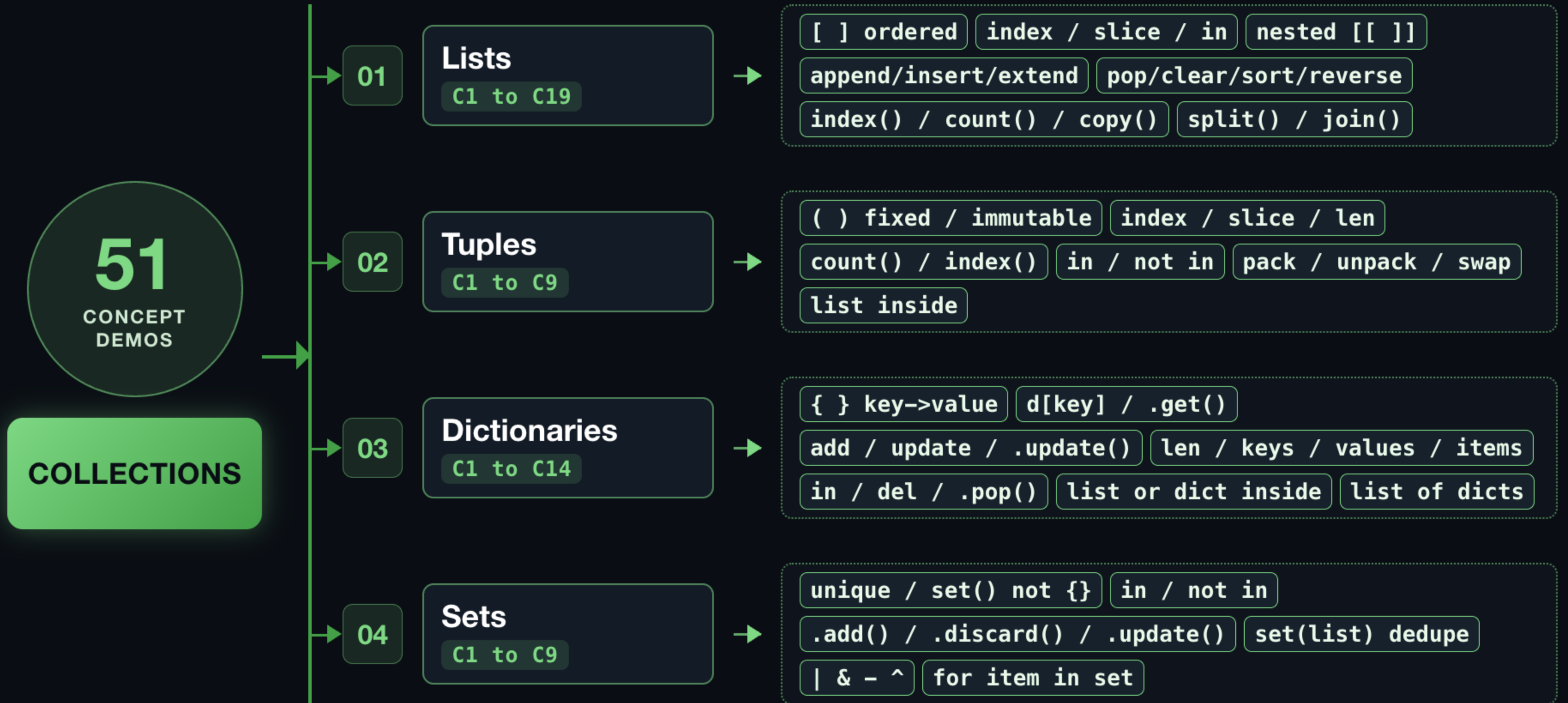
Section 04: Operators, Booleans, and Conditionals **Concept Map**

The full path through this section - we open here before each concept, then dive in.



Section 05: Collections **Concept Map**

The full path through this section - we open here before each concept, then dive in.



Section 06: Loops and Comprehensions **Concept Map**

The full path through this section - we open here before each concept, then dive in.



Section 07: Functions **Concept Map** (1 of 2)

The full path through this section - we open here before each concept, then dive in.



Section 07: Functions **Concept Map** (2 of 2)

The full path through this section - we open here before each concept, then dive in.



Milestone Project-1: What You Build

A real program you run from the terminal, built with **dictionaries and functions** and nothing you have not already met in Sections 01 to 07.

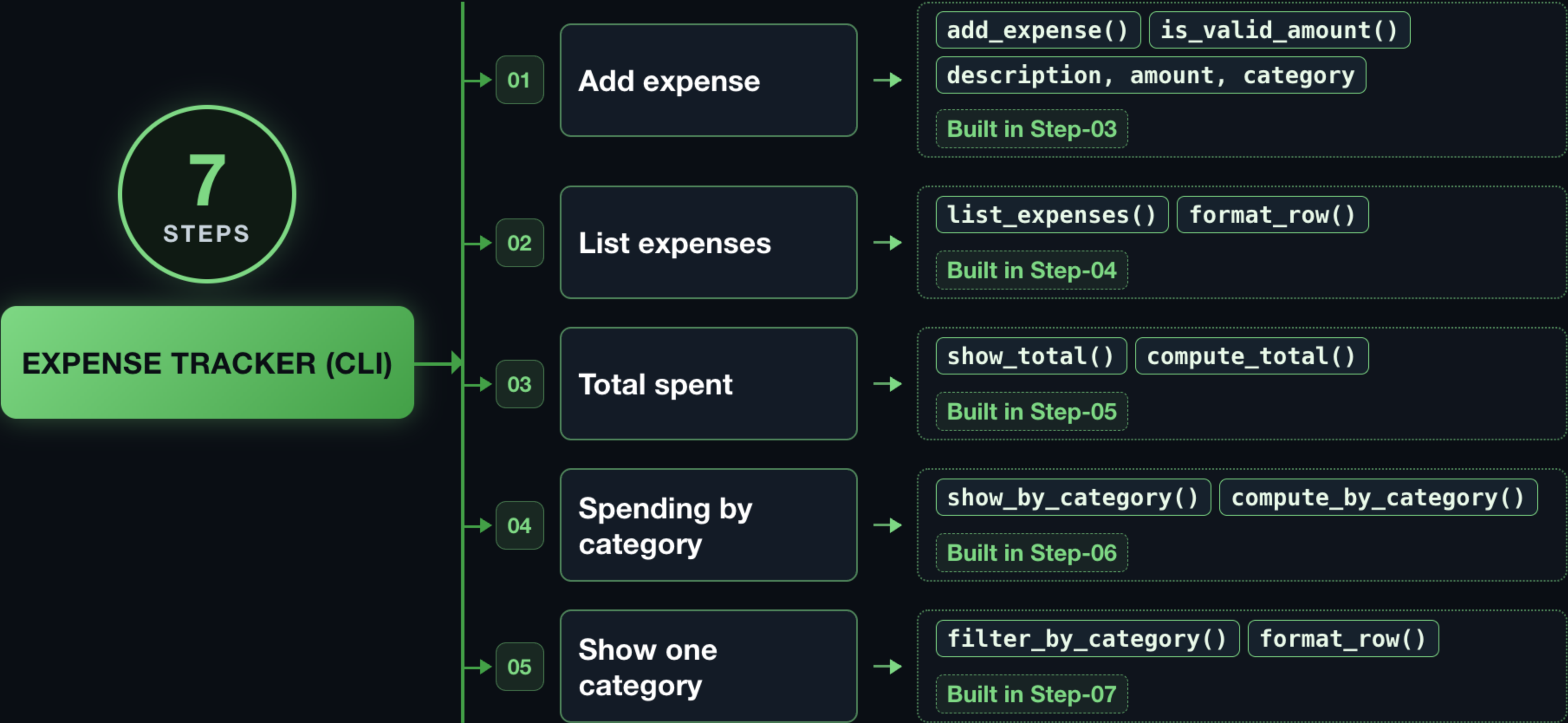
ON SCREEN	BEHIND IT	WHAT YOU WRITE IN PROJECT 1
Expense Tracker ==== Expense Tracker ==== 1) Add expense 2) List expenses 3) Total spent 4) Spending by category 5) Show one category 6) Quit Choose 1-6: _ Six choices, and you write every one of them	One expense	{ "amount": 3.5 } a dictionary with three keys
	All of them	[...] a plain list, held in main()
	The five actions	f5_actions.py one function per menu choice
	The helpers	f1 · f2 · f3 · f4 validate, format a row, total, totals by category

PROJECT 1

After Section 07, in 7 build steps

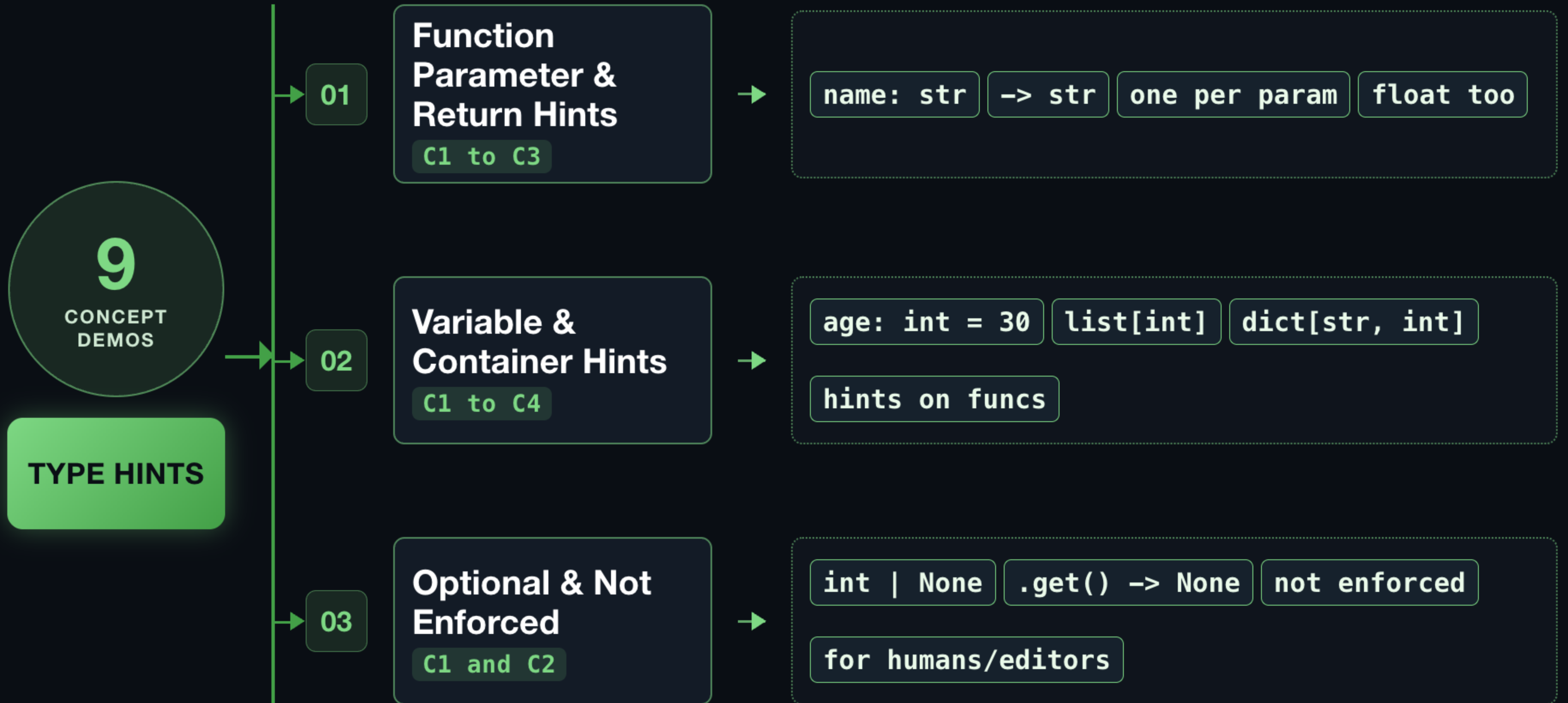
Milestone Project-1: CLI Expense Tracker - What You Will Build?

The five menu actions, and the functions that make each one work.



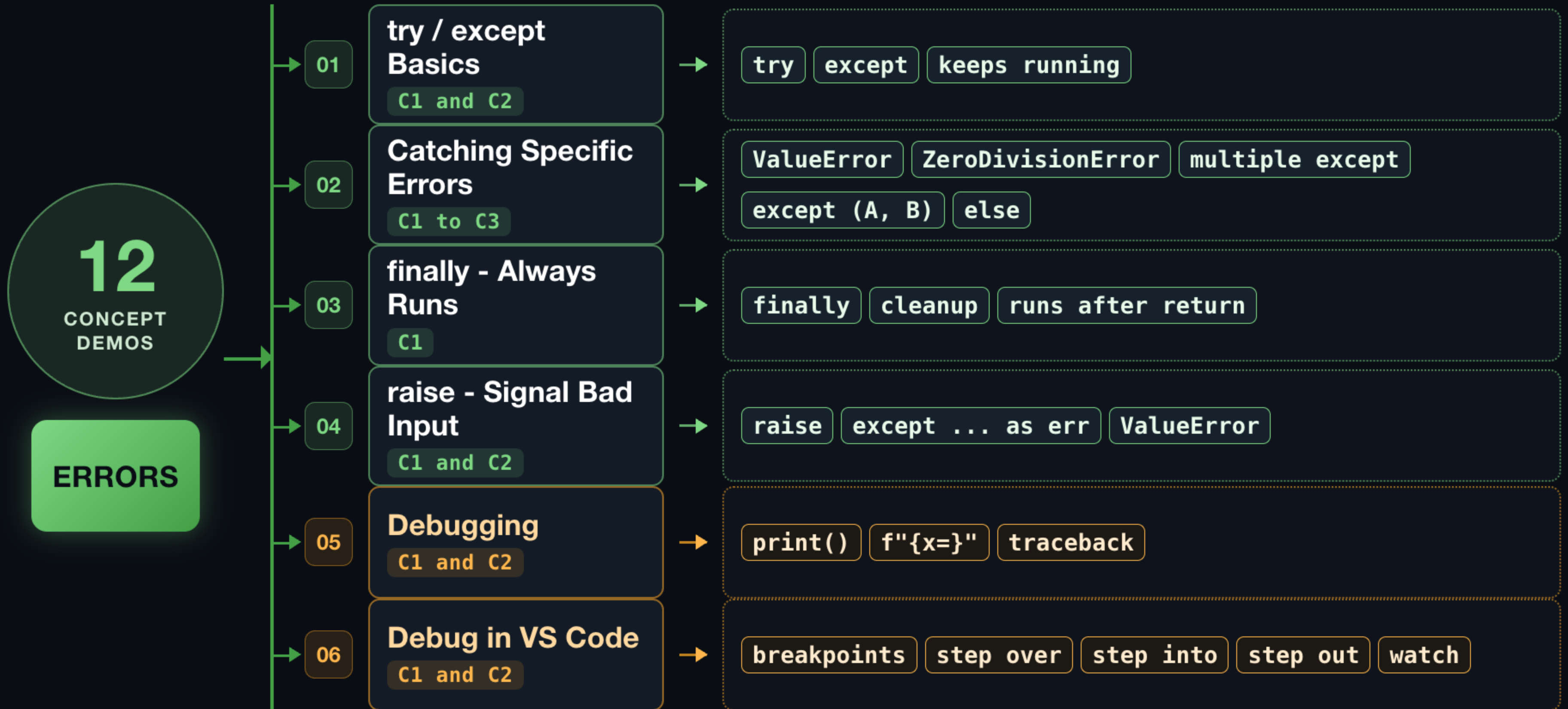
Section 08: Type Hints **Concept Map**

The full path through this section - we open here before each concept, then dive in.



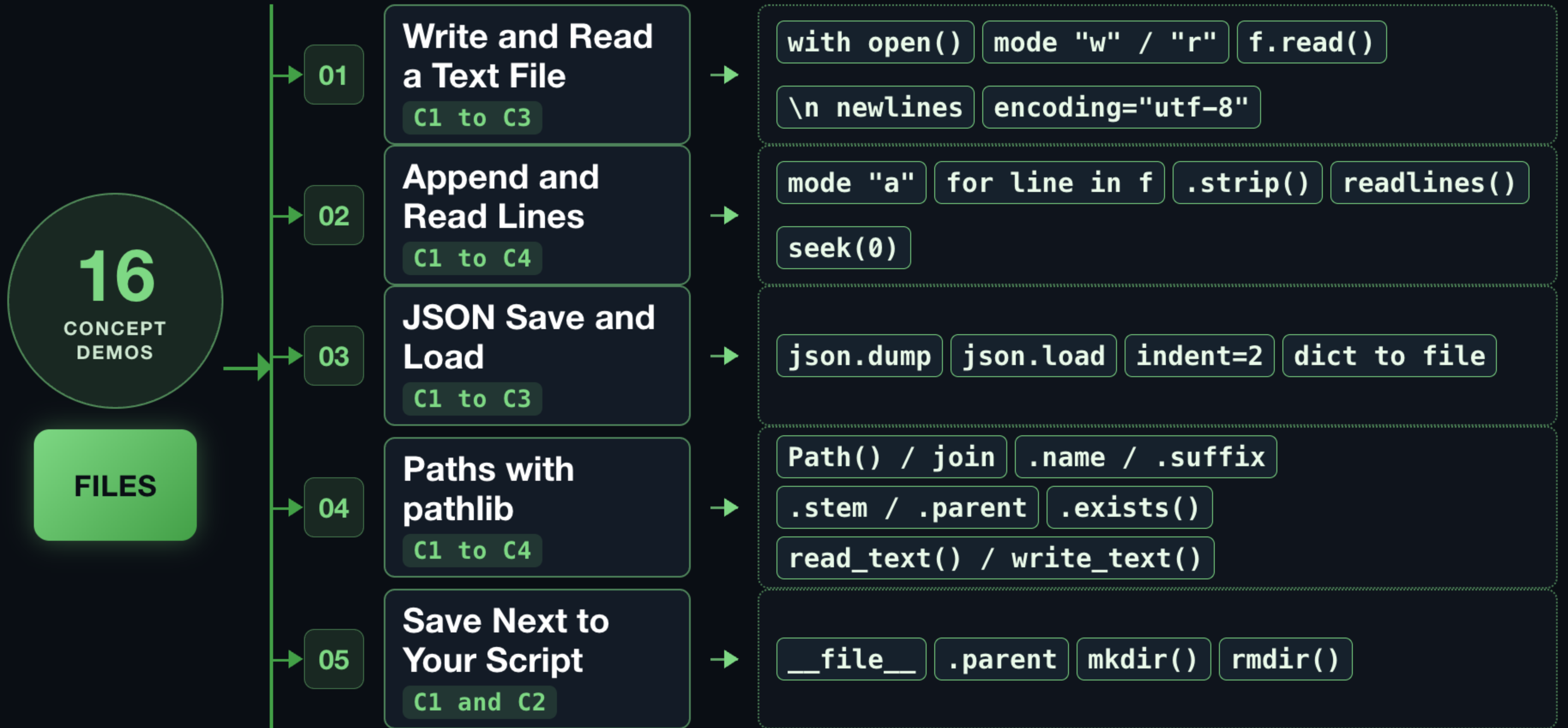
Section 09: Error Handling and Debugging Concept Map

The full path through this section - we open here before each concept, then dive in.



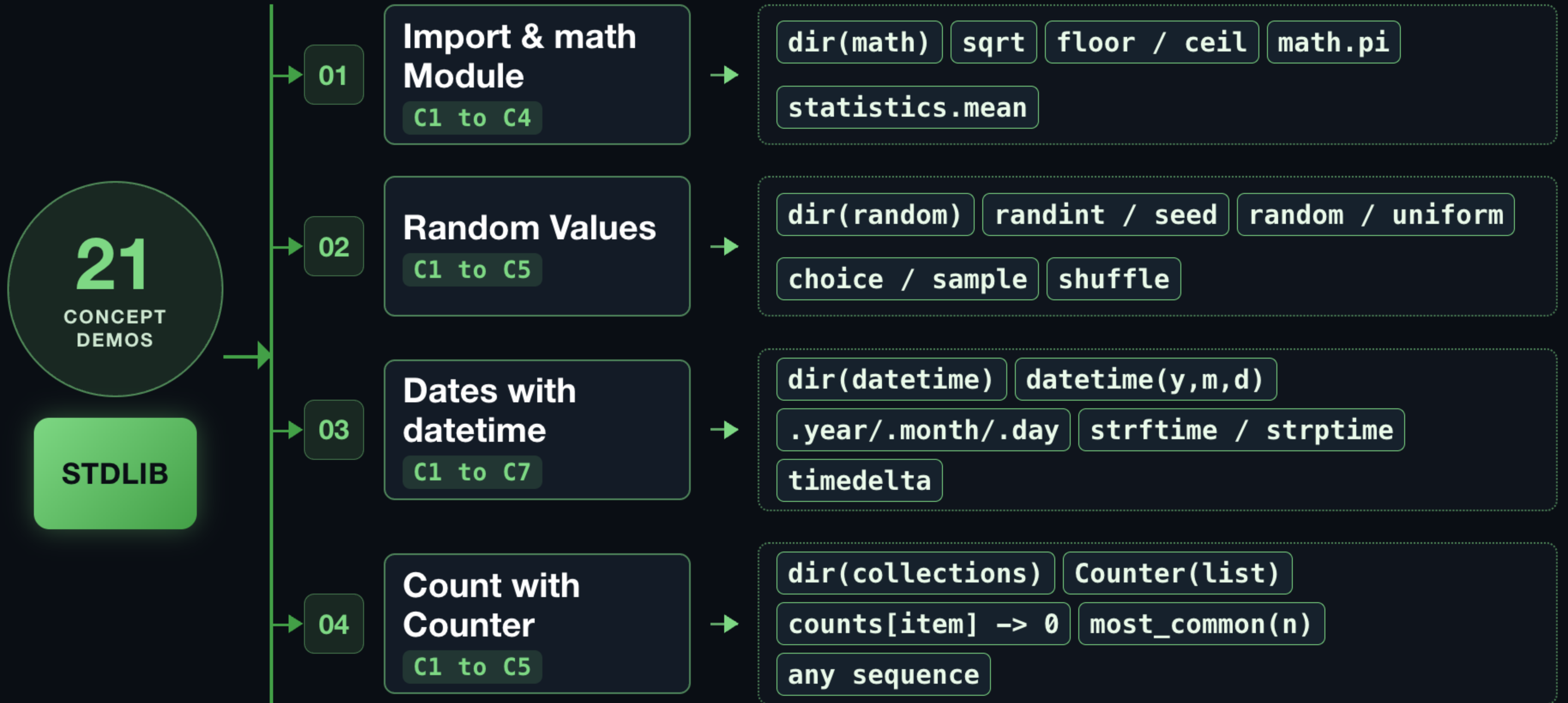
Section 10: Files, JSON, and Paths **Concept Map**

The full path through this section - we open here before each concept, then dive in.



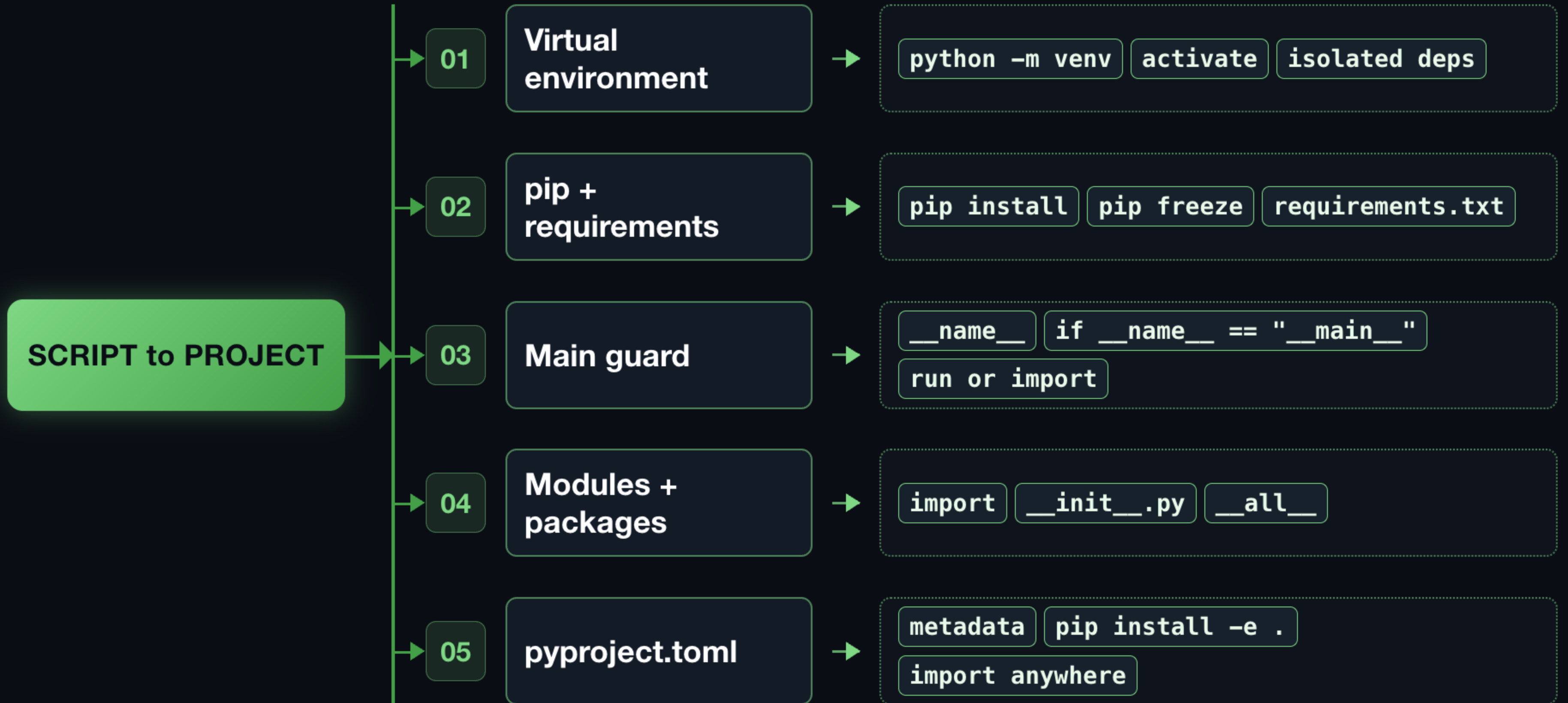
Section 11: Standard Library Essentials **Concept Map**

The full path through this section - we open here before each concept, then dive in.



Section 12: Virtual Environments and Packaging **Concept Map**

The path through Section 12: the habits that turn loose scripts into an isolated, installable project.



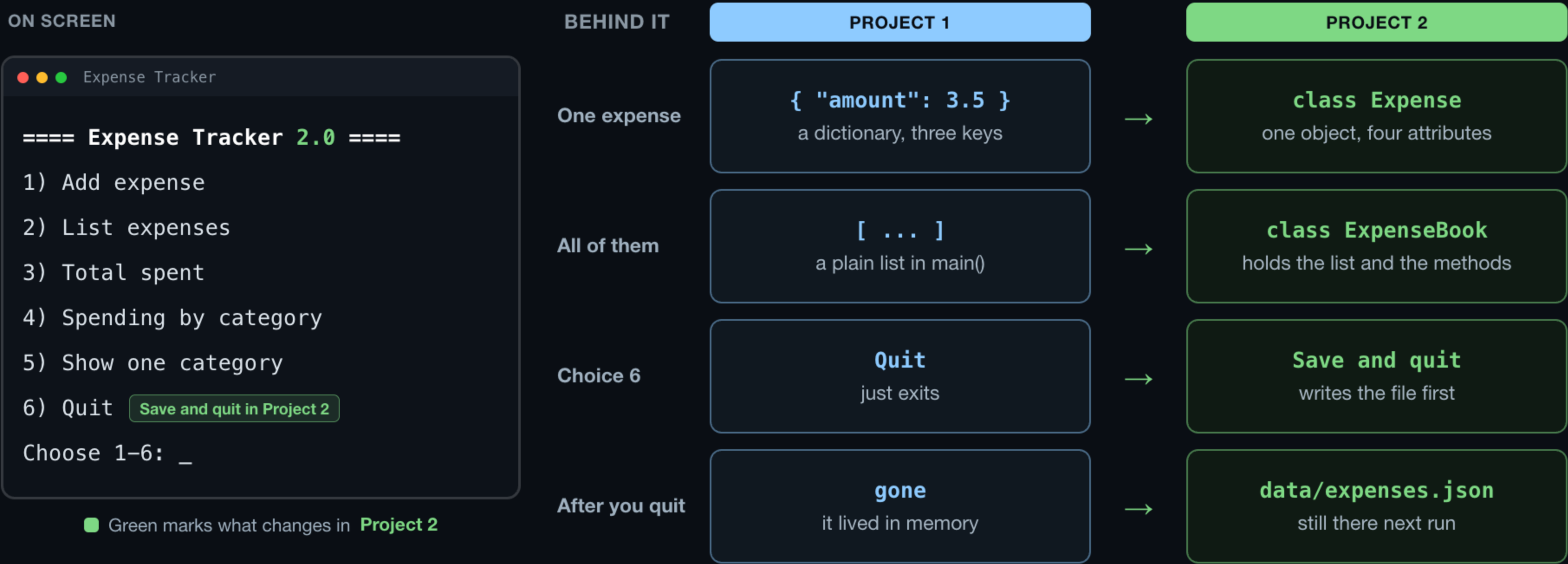
Section 13: Classes and OOP Basics Concept Map

The full path through this section: we open here before each concept, then dive in.



Milestone Projects 1 and 2: What You Build

One app, built twice. Project 1 with dictionaries and functions. Project 2 **rebuilt with classes**, with its data saved to a file.



PROJECT 1

After Section 07, in 7 build steps

PROJECT 2

After Section 13, in 11 build steps

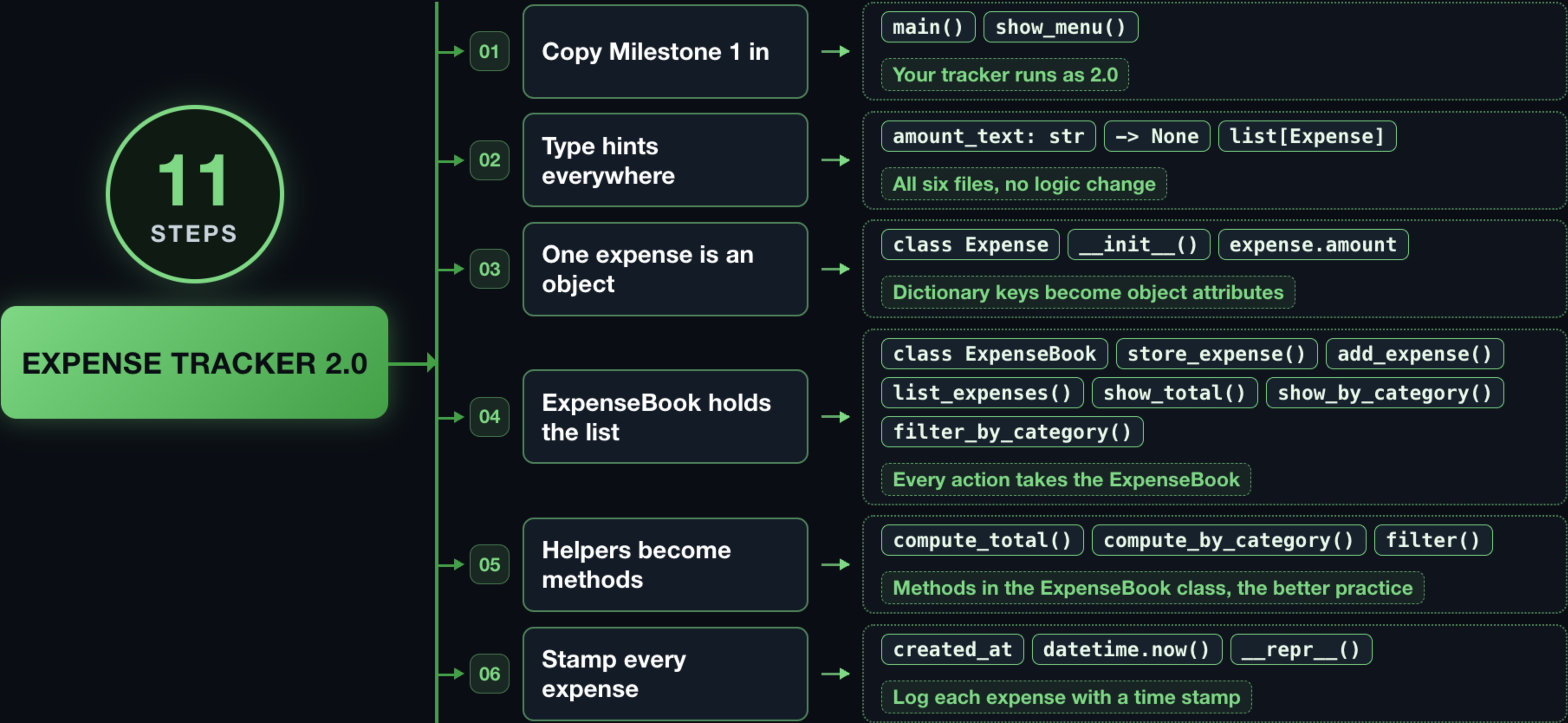
Milestone Project-2: Upgrade CLI Expense Tracker to Persistent Expense Tracker

The same Expense Tracker and the same menu. Milestone 2 changes how it works inside, using Sections 08 to 13.

	MILESTONE 1: DICTIONARY-BASED	UPGRADE	MILESTONE 2: OBJECTS + JSON + ERROR HANDLING
One expense	a dictionary with three keys	Section 13 →	an <code>Expense</code> object with attributes
Who owns the list	<code>main()</code> holds <code>expenses = []</code>	Section 13 →	<code>ExpenseBook</code> owns it: <code>book = ExpenseBook(...)</code>
Read a value	<code>expense["amount"]</code>	Section 13 →	<code>expense.amount</code>
Total spent	<code>compute_total(expenses)</code>	Section 13 →	<code>book.compute_total()</code>
By category	<code>compute_by_category(expenses)</code>	Section 13 →	<code>book.compute_by_category()</code>
One category	a list comprehension inside <code>filter_by_category()</code>	Section 13 →	<code>book.filter()</code> , called by the same action
Types	<i>no type hints</i>	Section 08 →	<code>amount_text: str, -> bool, list[Expense]</code>
Quit and restart	everything is gone	Section 10 →	<code>book.save()</code> and <code>book.load(): data/expenses.json</code>
A broken file	<i>no file, so nothing to break</i>	Section 09 →	<code>try / except</code> : a warning, no crash
New answers	<i>not there yet</i>	Section 11 →	<code>created_at</code> time stamp on every expense
One package	<i>nothing but Python</i>	Section 12 →	<code>.venv</code> and <code>pip install rich</code> : a real table (<i>bonus</i>)

Milestone Project-2: CLI Persistent Expense Tracker - What You Will Build? 1 of 2

Eleven steps build it. Steps 01 to 06 turn your Milestone 1 code into objects and a book.



Milestone Project-2: CLI **Persistent** Expense Tracker - What You Will **Build?** 2 of 2

Steps 07 to 11 give it a file, a guard, a full run, and the optional rich bonus.

